

Title:

To know about line coding & various types of line coding

Objectives:

The primary objectives of this lab report are as follows:

- To understand line coding of Unipolar, Polar, Bipolar, and Manchester Encoding using Google Collaboratory and NumPy.
- To plot the wave form for different line coding.
- Additionally evaluate the spectral characteristics, bandwidth requirements, and error detection capabilities of these coding methods.

Theory:

Line coding is the process of converting binary data into a specific pattern of electrical pulses. Different line coding schemes offer various advantages and are chosen based on the application requirements. The primary line coding schemes explored in this experiment are:

1. Unipolar Encoding (NRZ):

Unipolar encoding represents binary data using a single voltage level.

Typically, binary '1' is assigned a high voltage, while binary '0' is either no voltage or a low voltage level. This method is simple to implement, but it suffers from synchronization issues and the presence of a DC component, which can cause signal distortion over long transmission distances.

2. Polar Encoding (NRZ):

Polar encoding eliminates the DC component by using both positive and negative voltage levels. Here, binary '1' is represented by a positive voltage, while binary '0' is represented by a negative voltage. This approach improves synchronization compared to unipolar encoding, but it still lacks inherent synchronization mechanisms, making it susceptible to errors in certain transmission conditions.

3. Bipolar Encoding (AMI):

Bipolar encoding, also known as Alternate Mark Inversion (AMI), enhances signal transmission by alternating the voltage polarity for binary '1', while binary '0' is represented by zero voltage. This alternating pattern reduces DC buildup and improves error detection since any deviation from the expected pattern signals an error. However, implementing this method is more complex compared to unipolar and polar encoding.

4. Manchester Encoding:

Manchester encoding integrates clock and data signals to provide better synchronization. Each bit period is divided into two halves, with a transition occurring in the middle. A high-to-low transition typically represents binary '1', while a low-to-high transition represents binary '0'. This encoding method ensures robust synchronization and helps minimize noise effects, but it requires higher bandwidth compared to other encoding schemes.

Procedure:

1. We generated digital data to be transmitted. .
2. Utilizing Python and NumPy, we implemented Unipolar, Polar, Bipolar, and Manchester encoding schemes.
3. We compared and contrasted the visual representation of Unipolar, Polar, Bipolar, and Manchester encoding

Implementation Steps:

1. We open MATLAB or Python on our personal computer to begin the experiment.
2. We create a random binary sequence, which serves as the raw data for encoding.

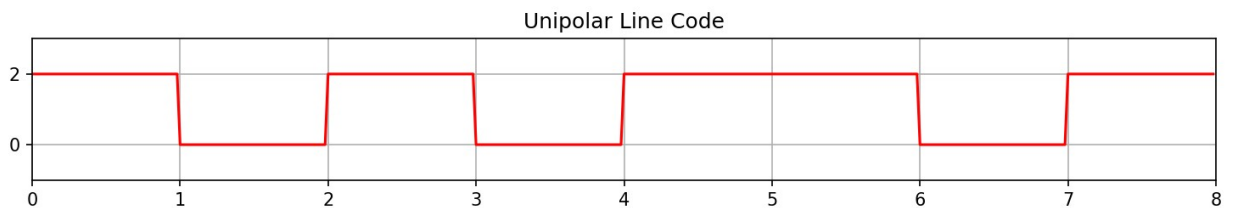
3. We write functions to encode the binary sequence using four different line coding schemes: Unipolar NRZ, Polar NRZ, Bipolar NRZ and Manchester Encoding
4. We visualize the encoded signals for each scheme to analyze their waveform characteristics.
5. We apply the Fourier Transform to our encoded signals to examine the frequency spectrum.
6. We evaluate and compare the bandwidth usage and spectral efficiency of each coding scheme to understand their performance.

Experimental Code & Graph:

Unipolar Line Coding:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  n = np.array([1, 0, 1, 0, 1, 1, 0, 1])
5
6  # Unipolar Line Coding
7  nn = np.where(n == 1, 2, 0)
8  t = np.arange(0, len(n), 0.02)
9  y = np.zeros_like(t)
10
11  i = 0
12  for j in range(len(t)):
13      if t[j] >= i + 1:
14          i += 1
15          if i < len(nn):
16              y[j] = nn[i]
17
18  plt.figure(figsize=(12, 8))
19  plt.subplot(4, 1, 1)
20  plt.plot(t, y, 'r-')
21  plt.title('Unipolar Line Code')
22  plt.grid(True)
23  plt.axis([0, len(n), -1, 3])
24  plt.show()
```

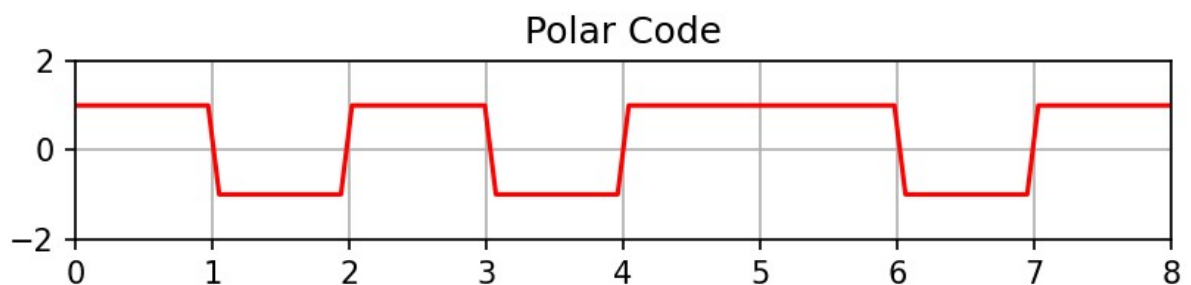
Output:



Polar Line Coding:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  n = np.array([1, 0, 1, 0, 1, 1, 0, 1])
5
6  # Polar Line Coding
7  nn = np.where(n == 1, 1, -1)
8  t = np.linspace(0, len(n), num=100)
9  y = np.zeros_like(t)
10
11  i = 0
12  for j in range(len(t)):
13      if t[j] >= i + 1 and i < len(nn) - 1:
14          i += 1
15      y[j] = nn[i]
16
17  plt.subplot(4, 1, 2)
18  plt.plot(t, y, 'r-')
19  plt.title('Polar Code')
20  plt.grid(True)
21  plt.axis([0, len(n), -2, 2])
22  plt.show()
```

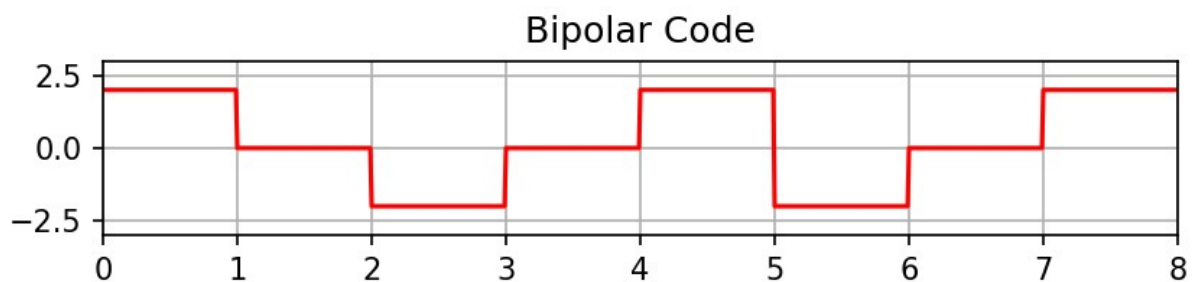
Output:



Bipolar Line Coding:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = np.array([1, 0, 1, 0, 1, 1, 0, 1])
5
6 # Bipolar Line Coding
7 nn = np.zeros_like(n, dtype=int)
8 sign = -1
9 for ii in range(len(n)):
10     if n[ii] == 1:
11         nn[ii] = -2 * sign
12         sign *= -1
13     else:
14         nn[ii] = 0
15
16 t = np.arange(0, len(n), 0.01)
17 y = np.zeros_like(t)
18 i = 0
19 for j in range(len(t)):
20     if t[j] >= i + 1:
21         i += 1
22         y[j] = nn[i]
23
24 plt.subplot(4, 1, 3)
25 plt.plot(t, y, 'r-')
26 plt.title('Bipolar Code')
27 plt.grid(True)
28 plt.axis([0, len(n), -3, 3])
29 plt.show()
```

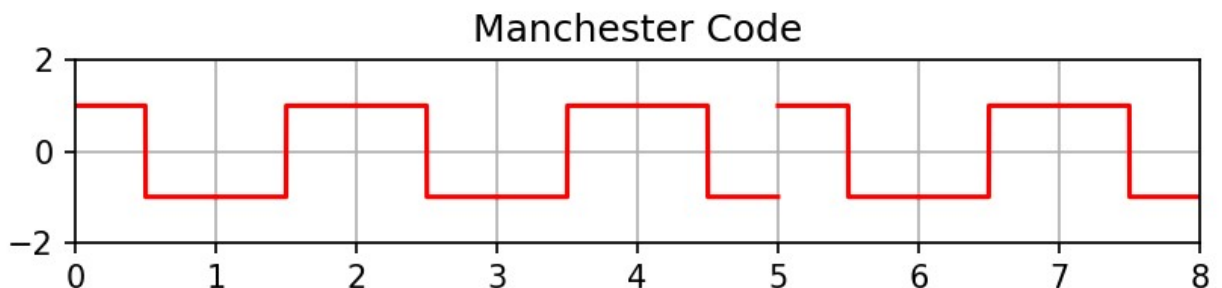
Output:



Manchester Line Coding:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  n = np.array([1, 0, 1, 0, 1, 1, 0, 1])
5
6  # Manchester Line Coding
7  plt.subplot(4, 1, 4)
8  for i in range(len(n)):
9      t = np.arange(i, i + 1, 0.001)
10     if n[i] == 1:
11         y = np.where((t - i) < 0.5, 1, -1)
12     else:
13         y = np.where((t - i) < 0.5, -1, 1)
14     plt.plot(t, y, 'r-')
15
16 plt.title('Manchester Code')
17 plt.grid(True)
18 plt.axis([0, len(n), -2, 2])
19 plt.show()
```

Output:



Discussion:

1. Through the experiment, we gained a solid understanding of **line coding techniques** used in digital communication. We analyzed Unipolar, Polar, Bipolar, and Manchester encoding, focusing on their **error rates, bandwidth efficiency, synchronization, and error detection**.
2. We familiarized ourselves with **NumPy and Matplotlib**, using them to generate, visualize, and analyze encoded signals effectively. These libraries helped us process signal data and compare different coding methods efficiently.
3. We explored the theoretical concepts behind each encoding scheme and **implemented them in a practical setting**. By encoding a binary sequence and observing its behavior under noisy conditions, we assessed the advantages and limitations of each method.
4. Initially, some parts of the **signal waveforms were not properly visualized**, especially in encoding transitions. We referred to **books and additional resources**, which helped us **sequence the signals correctly**, ensuring proper modulation and demodulation in the encoding process.